Kafka Stream: KStream, KTable, GlobalKTable

Saturday, 27 June 2026 10:25 AM

Pre-requisite:



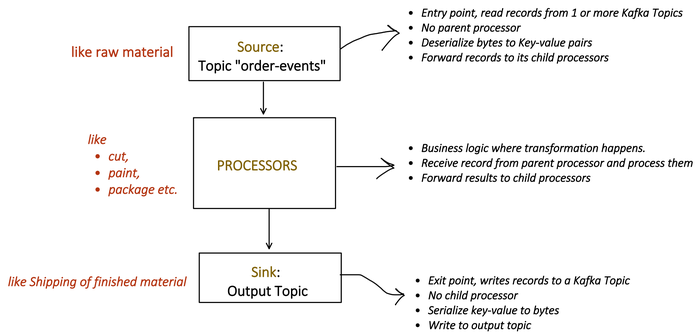
If we are clear with Kafka Stream Architecture, then we already know that the major part in Kafka Stream is writing TOPOLOGY.
Rest is all managed by framework.

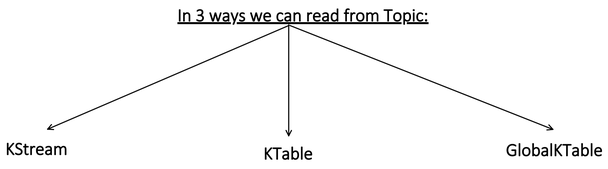
TOPOLOGY has 3 parts:

- Source
- Processor
- Sink

Today, lets focus on "SOURCE".

- Source is the entry point of a Kafka Streams topology.
- It reads records from one or more Kafka topics and forwards them to the stream processing pipeline.





KStream:

Each record read from the source topic partitions, is treated as an independent event.

Example:

Lets say below are the events coming from the source partitions:

```
offset 0: { orderId: "001", status: "CREATED" }
offset 1: { orderId: "001", status: "SHIPPED" }
offset 2: { orderId: "001", status: "DELIVERED" }
```

Notice that all 3 events belongs to 1 orderId ="001" only. But KStream sees them as 3 separate events and forwards each event to stream pipeline.
So,
Use KStream, when each record matters.

StreamsBuilder, helps to create topology.
When we added @EnableKafkaStreams, Spring boot inject it into our @Bean method.

```
@Configuration
public class SourceTypesTopology {

    @Bean
    public void kstreamDemo(StreamsBuilder builder) {


        KStream<String, String> orders = builder.stream("order-events",
                                            Consumed.with(Serdes.String(),
                                            Serdes.String())
                                            );


        orders.peek((key, value) ->
                        System.out.println("key=" + key + " value=" + value));

        orders.to("order-events-output",
                                        Produced.with(Serdes.String(), Serdes.String()));
    }
}
```

.stream() method is used to create KStream.

Parameter it takes:

- *Source Topics from which records need to be consumed. Below is an example to read from multiple topics:*

```
builder.stream(
        List.of("order-events", "payment-events"),
        Consumed.with(Serdes.String(), Serdes.String())
);
```

- *Key and Value SerDes, as data in Kafka Topics are stored in bytes, so it must convert those bytes into Java Object.*

These are processor notes and SINK nodes, which we will see later.

KTable:

- KTable represents latest state of each key.
- It stores the topic in RocksDB and continuously updates the value associated with each key.

Example:

```
offset 0: { orderId: "001", status: "CREATED" }
offset 1: { orderId: "001", status: "SHIPPED" }
offset 2: { orderId: "001", status: "DELIVERED" }
```

Notice that all 3 events belongs to 1 orderId ="001" only.

KTable read event one by one.

- Reads: offset 0: { orderId: "001", status: "CREATED" }
    - Update the RocksDB
    - Also forwards the event to stream pipeline
- Reads: offset 1 { orderId: "001", status: "SHIPPED" }
    - Update the RocksDB
    - Also forwards the event to stream pipeline
- Reads: offset 2 { orderId: "001", status: "DELIVERED" }
    - Update the RocksDB

RocksDB

Key	Value
orderId: "001"	{ orderId: "001", status: "~~CREATED~~ ~~SHIPPED~~ DELIVERED" }

- Also forwards the event to stream pipeline

.table() method is used to create KTable.
Parameter it takes:
- *Source Topic from which records need to be consumed.*
- *Key and Value SerDes, as data in Kafka Topics are stored in bytes, so it must deserialize (convert those bytes into Java Object)*
- *Materialized object:*
  - *RocksDB store Name*
  - *RocksDB store data in Bytes form, so we need Serializer*
    - *Key Serializer*
    - *Value Serializer*

```
@Configuration
public class SourceTypesTopology {

    @Bean
    public void kTableDemo(StreamsBuilder builder) {


            KTable<String, String> orders = builder.table("order-events",
                                                Consumed.with(Serdes.String(), Serdes.String()),
                                                Materialized
                                                        .<String, String, KeyValueStore<Bytes, byte[]>>as("order-events-store")
                                                        .withKeySerde(Serdes.String())
                                                        .withValueSerde(Serdes.String())
                                                );


            orders .toStream()
                    .peek((key, value) ->
                            System.out.println("key=" + key + " value=" + value))
                    .to("order-events-output",
                            Produced.with(Serdes.String(), Serdes.String()));
    }
}
```

These are processor notes and SINK nodes, which we will see later.

Now couple of doubt might come:

1st doubt:
What's the point of storing the event latest state in RocksDB, when KTable also send each event forwards to stream pipeline like KStream.

There are 2 major advantages:

#1. When some other operation in the stream pipeline need to look up.

Example:

Use-case: without KTable

KStream: Order topic
KStream: Customer topic

T1: Customer record arrives:
key="SJ", value="Delhi"

T2: Customer record arrives:
key="SJ", value="Bangalore"

T3: Order arrives:
key="SJ", value="laptop"

and **join** with Customer record, so that we can have customer city too along with order details.

At T3, the join needs SJ's CURRENT city.

Without RocksDB, where would it look? The customer records already flowed past. They're gone. There's nothing to look up.

Use-case: with KTable

KStream: Order topic
KTable: Customer topic

T1: Customer record arrives:
key="SJ", value="Delhi"

T2: Customer record arrives:
key="SJ", value="Bangalore"

T3: Order arrives:
key="SJ", value="laptop"

RocksDB

Key	Value
SJ	Delhi

RocksDB

Key	Value
SJ	~~Delhi~~ Bangalore

and **join** with Customer record, so that we can have customer city too along with order details.

At T3, the join needs SJ's CURRENT city.

With RocksDB, we can easily find the city "**Bangalore**" and enriched our order details with it.

#2. Interactive Queries:

One big advantage of RocksDB is that, we can expose REST API endpoint, that query this local state store and return the current state without querying external database.

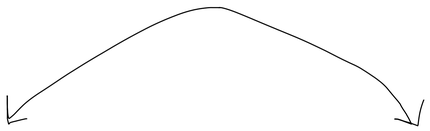
2nd doubt:

Say, in example which we have taken above:

Use-case: with KTable

KStream: Order topic
KTable: Customer topic

Does KTable partition events is given priority over KStream? Because Order event need to do join with Customer data stored in RocksDB?
So what if Order event comes before Customer RocksDB state is created?



Fresh Startup:
(No changelog present, first ever startup)

KStream and KTable source records arrive in parallel
→ Processed record by record in the order they arrive
→ NO priority given to KTable over KStream
→ If an order arrives BEFORE its customer, the join gets null

Restoration or Rebalance:
(RocksDB files on disk are gone but changelog topic present)

Before processing ANY new records:
→ Task reads the CHANGELOG topic (not the source topic)
→ Rebuilds RocksDB from changelog
→ Only THEN starts processing new records from source topics

This is NOT "read KTable source first" — it's "rebuild state store from backup"

3rd doubt:

Then join scenario which we have taken in above example,

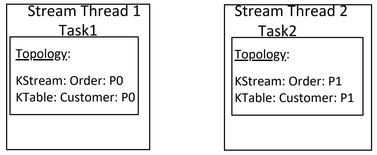
Use-case: with KTable

KStream: Order topic
KTable: Customer topic

for each Order event, we need to join with Customer

Is this even possible? Because say Customer Topic has 2 partitions then say:

- Customer Partition P0 goes in "task1" of StreamThread-1
- Customer Partition P1 goes in "task2" of StreamThread-2



So what if , order event in Task1 need to lookup for customer with id "SJ" which is present in Task2?

That's a very valid question:

So KStream and KTable works seamlessly only when:

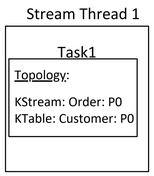
- No of partitions in both ORDER topic and CONSUMER Topic are same.
- And partition strategy (like Key) is same for both ORDER and CONSUMER topic.

This is called
Co-partitioning

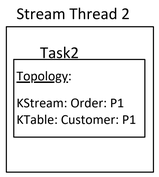
So that,

orders topic (2 partitions, key = customerId):
P0: SJ -> laptop, SJ -> phone
P1: RJ -> tablet, RJ -> mouse

customers topic (2 partitions, key = customerId):
P0: SJ -> "Delhi"
P1: RJ -> "Bangalore"



orders for SJ + customer record for SJ -> JOIN WORKS



orders for RJ + customer record for RJ -> JOIN WORKS

But when Co-partitioning is not there, then KStream and KTable join will not work.

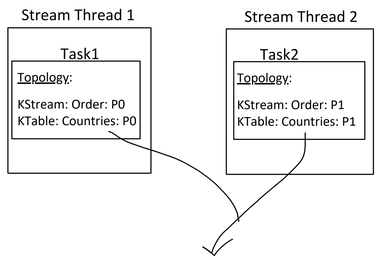
That's where:

GlobalKTable:

Problem with KTable:

Imagine in a kafka Stream Application, we have 2 stream Thread and we have
- orders topic with 2 partitions
- countries topic with 2 partitions

And each order event has country-id from that I need to join the countries and find the country name.



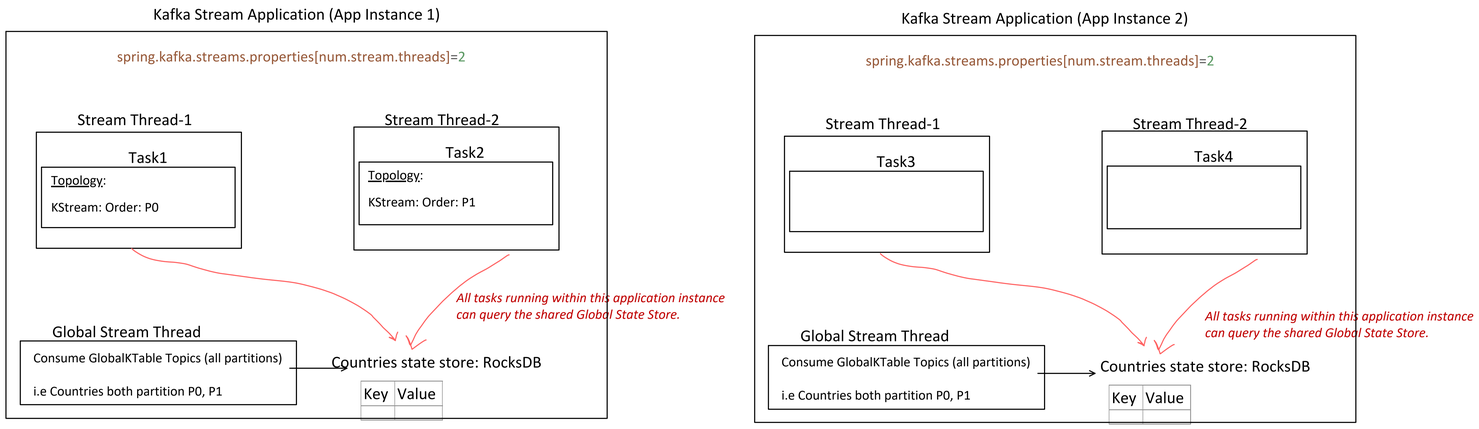
Notice that, Countries partitions are divided among multiple tasks.

So Task1 -> gets Countries Partition P0 -> sees SOME countries
Task2 -> gets Countries Partition P1 -> sees OTHER countries

So possible that, order event in TASK-1 might need a country that's on TASK-2, which cause JOIN FAILS

That's where GlobalKTable comes into picture

- Each Kafka Stream Application gets all partitions data of GlobalKTable topic



- Always use GlobalKTable for relatively small or reference datasets, because every Kafka Streams application instance maintains a complete local copy of the topic in its Global State Store.

Good candidates for GlobalKTable

- Countries
- Currency codes
- Configuration

etc.

Note:

On startup:

1. Global Stream Thread reads ALL partitions of "countries" topic (GlobalKTable topics)
2. Builds the global RocksDB store.
3. There is no changelog maintained for global RocksDB Store. Reason GlobalKTable is meant to hold small dataset only. So during startup, it read full topic (all partitions) itself.
4. ONLY AFTER this is complete, stream threads start

This is a "read first" behavior - but only for GlobalKTable, not for regular KTable.

Application Start

↓

Restore or create GlobalKTable first

↓

Start StreamThreads

.globalTable() method is used to create globalKTable.

Parameter it takes:

- *Source Topic from which records need to be consumed.*
- *Key and Value SerDes, as data in Kafka Topics are stored in bytes, so it must deserialize (convert those bytes into Java Object)*
- *Materialized object:*
  - *RocksDB store Name*
  - *RocksDB store data in Bytes form, so we need Serializer*
    - *Key Serializer*
    - *Value Serializer*

```
@Configuration
public class SourceTypesTopology {

    @Bean
    public void globalKTableDemo(StreamsBuilder builder) {

        GlobalKTable<String, String> countries = builder.globalTable("countries",
                                                    Consumed.with(Serdes.String(), Serdes.String()),
                                                    Materialized
```

```
                                                .<String, String, KeyValueStore<Bytes, byte[]>>as("countries-store")
                                                .withKeySerde(Serdes.String())
                                                .withValueSerde(Serdes.String())
                                        );


        KStream<String, String> ordersWithCountry = builder.stream("order-events",
                                                        Consumed.with(Serdes.String(), Serdes.String()));

        KStream<String, String> enriched = ordersWithCountry.join(
                                                        countries,
                                                        (orderId, orderJson) -> {
                                                            try {
                                                                JsonNode order = objectMapper.readTree(orderJson);
                                                                return order.get("countryCode").asText();
                                                            } catch (Exception e) {
                                                                //exception handling
                                                            }
                                                        },
                                                        (orderJson, countryName) ->
                                                                orderJson + " | country=" + countryName
                                                );

        enriched.peek((key, value) ->
                                System.out.println("key=" + key + " value=" + value))
        .to("order-enriched-with-country",
                                Produced.with(Serdes.String(), Serdes.String()));

         }
}
```

Joins are part of processor notes, which we will see later